UQA Python API Reference

This document covers the Python API for UQA (Unified Query Algebra), including the Engine, the fluent QueryBuilder API, core types, text analysis, storage, graph operations, and scoring modules.

UQA Python API Reference

- 1. Engine
  - Constructor
  - SQL Interface
  - Fluent QueryBuilder
  - Document Management
  - Graph Management
  - Named Graphs
  - Analyzer Management
  - Scoring Parameters
  - Calibration and Parameter Learning
  - Deep Learning
    - Training
    - Inference
    - SQL Equivalent
    - Model Persistence
  - Path Index Management
  - Graph Delta Operations
  - Transactions
  - Context Manager
- 2. SQLResult
  - Properties
  - Methods
  - Example
- 3. QueryBuilder (Fluent API)
  - Term Retrieval
  - Vector Search
  - Boolean Algebra
  - Filtering
  - Scoring
  - Advanced Scoring
  - Fusion
  - Advanced Fusion
  - Graph Operations
  - Temporal Graph Operations
  - Vector Exclusion
  - Joins
  - Aggregation
  - Hierarchical Data
  - Execution
- 4. Core Types
  - PostingEntry
  - Payload
  - GeneralizedPostingEntry
  - Vertex
  - Edge
  - IndexStats

- Predicates
5. PostingList
    - Boolean Algebra
    - Methods
    - Iteration
6. Text Analysis
    - Analyzer
    - Built-in Analyzers
    - Tokenizers
    - Token Filters
    - Character Filters
    - Analyzer Registry
    - Serialization
7. Storage Layer
    - DocumentStore
    - InvertedIndex
    - SpatialIndex (R*Tree)
    - HNSWIndex (Vector Index)
8. Graph Store
    - Graph Indexes
    - SubgraphIndex
    - Incremental Pattern Matching
    - Graph Patterns
9. Scoring
    - BM25
    - Bayesian BM25
    - Vector Scoring
    - Calibration Metrics
    - Parameter Learner
    - External Prior Scorer
    - Multi-Field Scorer
10. Query Planning
    - Optimizer
    - Cost Model
    - Cardinality Estimator
    - Join Order Optimizer (DPccp)
    - Plan Executor
11. Operators
    - Primitive Operators
    - Boolean Operators
    - Graph Operators
    - Graph Centrality Operators
    - Bounded RPQ
    - Weighted Path Query
    - Advanced Scoring Operators
    - Advanced Fusion Operators
    - ProgressiveFusionOperator
    - Temporal Graph Operators
    - GNN Operators
    - Graph Maintenance
    - Fusion Classes
    - Fusion Gating
        - `fuse_mean(probabilities: list[float]) -> float`
    - Hierarchical Operators
    - Aggregation Monoids

`QuantileMonoid(quantile: float = 0.5)`
Join Operators
Execution Context
12. Transaction
Methods
Properties
Usage
13. Table Schema
ColumnDef
ColumnStats
ForeignKeyDef
14. Foreign Data Wrappers
ForeignServer
ForeignTable
FDWPredicate
FDWHandler
Full Query Pushdown
Mixed Foreign-Local Queries
Automatic Fallback
Drop Statements
15. Complete Example

---

1. Engine

The `Engine` class is the main entry point for all UQA operations.

```
from uqa.engine import Engine
```

Constructor

```
Engine(
    db_path: str | None = None,
    parallel_workers: int = 4,
    spill_threshold: int = 0,
)
```

Parameter	Description
`db_path`	Path to SQLite file for persistence. `None` for in-memory.
`parallel_workers`	Number of parallel execution workers.
`spill_threshold`	Memory threshold (bytes) for disk spilling. 0 disables.

SQL Interface

```
engine.sql(query: str, params: list[Any] | None = None) -> SQLResult
```

Execute a SQL query. Returns an `SQLResult` object with `.columns` and `.rows`. Supports parameterized queries with `$1`, `$2`, etc.

```
result = engine.sql("SELECT * FROM users WHERE age > $1", [25])
for row in result.rows:
    print(row["name"], row["age"])
```

Fluent QueryBuilder

```
engine.query(table: str) -> QueryBuilder
```

Create a `QueryBuilder` scoped to a table for fluent API queries.

```
results = (
    engine.query(table="articles")
    .term("attention", field="body")
    .score_bm25("attention")
    .execute()
)
```

Document Management

```
engine.add_document(
    doc_id: DocId,
    document: dict[str, Any],
    table: str,
    embedding: NDArray | None = None,
) -> None
```

Add a document to a table's storage and indexes.

```
engine.get_document(doc_id: DocId, table: str) -> dict[str, Any] | None
```

Retrieve a document by its ID from the given table. Returns the stored document dict, or `None` if the document does not exist.

```
engine.delete_document(doc_id: DocId, table: str) -> None
```

Remove a document from a table's storage and indexes.

Graph Management

```
engine.add_graph_vertex(vertex: Vertex, table: str) -> None
engine.add_graph_edge(edge: Edge, table: str) -> None
engine.get_graph_store(table: str) -> GraphStore
```

Add vertices and edges to a table's graph store. `get_graph_store()` returns the `GraphStore` associated with a table for direct vertex lookups and index building.

Named Graphs

```
engine.create_graph(name: str) -> GraphStore
engine.drop_graph(name: str) -> None
engine.get_graph(name: str) -> GraphStore
engine.has_graph(name: str) -> bool
```

Manage named graphs for Cypher queries and FROM-clause graph functions. Named graphs are persisted via the SQLite catalog (`_graph_catalog` and `_graph_membership` tables). All graph functions now accept direct graph names without the `graph:` prefix (backward compatible):

```
SELECT * FROM traverse(1, 'knows', 2, 'social');
SELECT * FROM rpq('knows/works_with', 1, 'social');
SELECT * FROM temporal_traverse(1, 'knows', 2, 150, 'net');
SELECT * FROM pagerank(0.85, 'social');
SELECT * FROM hits(20, 'social');
SELECT * FROM betweenness('social');
```

Analyzer Management

```
engine.create_analyzer(name: str, config: dict[str, Any]) -> None
engine.drop_analyzer(name: str) -> None
engine.set_table_analyzer(table_name: str, field: str, analyzer_name: str, phase: str = "both") -> None
engine.get_table_analyzer(table_name: str, field: str, phase: str = "index") -> Any
```

Create named analyzers and assign them to table fields for custom tokenization.

The `phase` parameter in `set_table_analyzer` controls when the analyzer is applied:

- `"index"` — used during document indexing (controls what tokens are stored)
- `"search"` — used during query processing (controls how query terms are expanded)
- `"both"` — assigned to both phases (default)

The `phase` parameter in `get_table_analyzer` selects which phase's analyzer to retrieve (`"index"` by default). The search-time analyzer falls back to the index-time analyzer, which falls back to the default analyzer.

```
# Dual analyzer: index without synonyms, search with synonym expansion
engine.set_table_analyzer("products", "body", "plain_analyzer", phase="index")
engine.set_table_analyzer("products", "body", "synonym_analyzer", phase="search")
```

Scoring Parameters

```
engine.save_scoring_params(name: str, params: dict[str, Any]) -> None
engine.load_scoring_params(name: str) -> dict[str, Any] | None
engine.load_all_scoring_params() -> list[tuple[str, dict[str, Any]]]
```

Persist and retrieve Bayesian calibration parameters for named scoring signals.

Calibration and Parameter Learning

```
# Calibration diagnostics
report = engine.calibration_report(table, field, query, labels)
# Returns dict with ECE, Brier score, etc.

# Batch parameter learning
learned = engine.learn_scoring_params(table, field, query, labels, mode="balanced")
# Returns {"alpha": float, "beta": float, "base_rate": float}

# Online parameter update
engine.update_scoring_params(table, field, score, label)
```

Deep Learning

Train and run inference with deep learning models analytically (no backpropagation). Training uses random-weight initialization with Bayesian ridge regression — the random weights act as a prior and ridge regression computes the posterior. Graph structure is exploited via message-passing convolution over spatial grid graphs.

Training

```
engine.deep_learn(
    model_name: str,
    table_name: str,
    label_field: str,
    embedding_field: str,
    edge_label: str,
    layer_specs: list[LayerSpec],
    gating: str = "none",
    lam: float = 1.0,
    l1_ratio: float = 0.0,
    prune_ratio: float = 0.0,
) -> dict[str, Any]
```

Parameter	Description
`model_name`	Name under which the trained model is persisted.
`table_name`	Table containing training data.
`label_field`	Column holding class labels.
`embedding_field`	Column holding input vectors (`VECTOR` type).
`edge_label`	Edge label in the graph connecting grid nodes (e.g. `"spatial"`).
`layer_specs`	Ordered list of layer specifications defining the network architecture.
`gating`	Activation gating: `"none"`, `"relu"`, or `"swish"`.
`lam`	Ridge regression regularization strength (lambda).
`l1_ratio`	Elastic net L1 ratio (0.0 = pure ridge). Values > 0 enable proximal gradient descent (ISTA), warm-started from the ridge solution.
`prune_ratio`	Magnitude pruning ratio (0.0 = no pruning). Values > 0 zero the smallest weights by absolute magnitude percentile after training.

The return dict contains `"training_accuracy"` and the full model configuration.

Layer specifications:

```
from uqa.operators.deep_learn import (
    ConvSpec, PoolSpec, FlattenSpec, GlobalPoolSpec,
    DenseSpec, SoftmaxSpec, AttentionSpec,
)
```

Spec	Parameters	Description
`ConvSpec(kernel_hops, n_channels, init_mode)`	`kernel_hops=1`, `n_channels=1`, `init_mode="kaiming"`	Graph convolution layer. `n_channels > 1` creates multi-channel feature maps. `init_mode`: `"kaiming"` (random normal), `"orthogonal"` (QR decomposition), `"gabor"` (structured filter bank), `"kmeans"` (data-dependent patch dictionary).
`PoolSpec(method, pool_size)`	`method="max"`, `pool_size=2`	Pooling layer — `"max"` or `"mean"`.
`FlattenSpec()`	(none)	Flatten spatial nodes into a single vector.
`GlobalPoolSpec(method)`	`method="avg"`	Channel-preserving spatial reduction. `"avg"`: per-channel mean, `"max"`: per-channel max, `"avg_max"`: concatenation of both (2x channels).
`DenseSpec(output_channels)`	`output_channels=10`	Dense (fully connected) layer.
`SoftmaxSpec()`	(none)	Softmax classification head.
`AttentionSpec(n_heads, mode)`	`n_heads=1`, `mode="content"`	Self-attention layer (Theorem 8.3, Paper 4). Modes: `"content"` (Q=K=V=X), `"random_qk"` (random Q,K, V=X), `"learned_v"` (random Q,K, supervised V).

Example:

```python
from uqa.engine import Engine
from uqa.operators.deep_learn import (
    ConvSpec, PoolSpec, FlattenSpec, GlobalPoolSpec,
    DenseSpec, SoftmaxSpec, AttentionSpec,
)

engine = Engine()

# Create table with labels and embeddings
engine.sql("""
    CREATE TABLE images (
        id SERIAL PRIMARY KEY,
        label TEXT NOT NULL,
        embedding VECTOR(784)
    )
""")
# ... insert data ...

# Build grid graph
engine.sql("SELECT * FROM build_grid_graph('images', 28, 28, 'spatial')")

# Train via Python API
result = engine.deep_learn(
    model_name="my_model",
    table_name="images",
    label_field="label",
    embedding_field="embedding",
    edge_label="spatial",
    layer_specs=[
        ConvSpec(n_channels=32),
        PoolSpec(method="max", pool_size=2),
        FlattenSpec(),
        DenseSpec(output_channels=10),
        SoftmaxSpec(),
    ],
    gating="relu",
    lam=1.0,
)
print(result["training_accuracy"])

# Train with self-attention and pruning
result = engine.deep_learn(
    model_name="attn_pruned_model",
    table_name="images",
    label_field="label",
    embedding_field="embedding",
    edge_label="spatial",
    layer_specs=[
        ConvSpec(n_channels=32),
        AttentionSpec(n_heads=4, mode="content"),
        PoolSpec(method="max", pool_size=2),
        FlattenSpec(),
        DenseSpec(output_channels=10),
        SoftmaxSpec(),
    ],
```

```
    gating="relu",
    lam=1.0,
    l1_ratio=0.3,
    prune_ratio=0.5,
)
print(result["training_accuracy"], result["weight_sparsity"])
```

Inference

```
engine.deep_predict(
    model_name: str,
    input_embedding: list[float],
) -> list[tuple[int, float]]
```

Run inference using a trained model. Returns a list of `(class_index, probability)` tuples sorted by probability descending.

```
predictions = engine.deep_predict("my_model", input_vector)
for class_idx, probability in predictions:
    print(f"  class {class_idx}: {probability:.4f}")
```

SQL Equivalent

The same training operation is available as a SQL aggregate function:

```
SELECT deep_learn(
    'my_model', label, embedding, 'spatial',
    convolve(n_channels => 32),
    pool('max', 2),
    flatten(),
    dense(output_channels => 10),
    softmax(),
    gating => 'relu', lambda => 1.0
) FROM images;

-- With self-attention and pruning
SELECT deep_learn(
    'attn_pruned_model', label, embedding, 'spatial',
    convolve(n_channels => 32),
    attention(n_heads => 4, mode => 'content'),
    pool('max', 2),
    flatten(),
    dense(output_channels => 10),
    softmax(),
    gating => 'relu', lambda => 1.0,
    l1_ratio => 0.3, prune_ratio => 0.5
) FROM images;
```

Model Persistence

Trained models are automatically saved to the catalog. Use `load_model()` and `delete_model()` to manage them:

```
engine.save_model(model_name: str, config: dict[str, Any]) -> None
engine.load_model(model_name: str) -> dict[str, Any] | None
engine.delete_model(model_name: str) -> None
```

```
# Retrieve a trained model's configuration
config = engine.load_model("my_model")

# Remove a model from the catalog
engine.delete_model("my_model")
```

Path Index Management

```
# Build path index for RPQ acceleration
engine.build_path_index(graph_name, [["knows"], ["knows", "works_with"]])

# Retrieve path index
idx = engine.get_path_index(graph_name)

# Drop path index
engine.drop_path_index(graph_name)
```

Graph Delta Operations

```
from uqa.graph.delta import GraphDelta

delta = GraphDelta()
delta.add_vertex(Vertex(1, "person", {"name": "Alice"}))
delta.add_edge(Edge(1, 1, 2, "knows"))

# Apply delta with version tracking and path index invalidation
version = engine.apply_graph_delta(graph_name, delta)
```

Transactions

```
engine.begin() -> Transaction
```

Start an explicit transaction. Returns a `Transaction` context manager.

```
with engine.begin() as tx:
    engine.sql("INSERT INTO t VALUES (1, 'a')")
    engine.sql("INSERT INTO t VALUES (2, 'b')")
    # auto-commits on exit; auto-rollbacks on exception
```

Context Manager

```
with Engine(db_path="data.db") as engine:
    engine.sql("SELECT 1")
# engine.close() called automatically
```

2. SQLResult

Returned by `engine.sql()`.

```
from uqa.sql.compiler import SQLResult
```

Properties

Property	Type	Description
`columns`	`list[str]`	Column names
`rows`	`list[dict[str, Any]]`	Row data as list of dicts

Methods

```
result.to_arrow() -> pa.Table       # Convert to PyArrow Table
result.to_parquet(path: str) -> None # Write to Parquet file
len(result)                          # Number of rows
str(result)                          # Formatted table output
```

Example

```
result = engine.sql("SELECT id, name FROM users ORDER BY id LIMIT 5")
print(result.columns)  # ['id', 'name']
for row in result.rows:
    print(row["id"], row["name"])

# Export to Arrow/Parquet
arrow_table = result.to_arrow()
result.to_parquet("users.parquet")
```

---

3. QueryBuilder (Fluent API)

The `QueryBuilder` provides a chainable, type-safe interface for building queries across all paradigms.

```
from uqa.api.query_builder import QueryBuilder
```

All methods return a new `QueryBuilder`, enabling method chaining.

Term Retrieval

```
.term(term: str, field: str | None = None) -> QueryBuilder
```

Retrieve documents containing a term. The query term is automatically analyzed using the same analyzer that indexed the field.

```
results = engine.query("articles").term("attention", field="body").execute()
```

Vector Search

```
.vector(query: NDArray, threshold: float, field: str = "embedding") -> QueryBuilder
.knn(query: NDArray, k: int, field: str = "embedding") -> QueryBuilder
```

Method	Description
`vector()`	All documents with similarity >= threshold
`knn()`	Top-k nearest neighbors

```
import numpy as np
q = np.random.randn(64).astype(np.float32)
results = engine.query("papers").knn(q, k=10).execute()
```

Boolean Algebra

```
.and_(other: QueryBuilder) -> QueryBuilder   # Intersection
.or_(other: QueryBuilder) -> QueryBuilder    # Union
.not_() -> QueryBuilder                      # Complement
```

```
text_q = engine.query("products").term("wireless")
vec_q = engine.query("products").knn(audio_vec, k=10)
combined = text_q.and_(vec_q)
results = combined.execute()
```

Filtering

```
.filter(field: str, predicate: Predicate) -> QueryBuilder
```

Filter results by a field predicate. See Predicates for all available predicate types.

```
from uqa.core.types import GreaterThanOrEqual, InSet

results = (
    engine.query("products")
    .term("laptop")
    .filter("price", GreaterThanOrEqual(500))
    .filter("category", InSet(frozenset({"electronics", "computers"})))
    .execute()
)
```

Scoring

```
.score_bm25(query: str, field: str | None = None) -> QueryBuilder
.score_bayesian_bm25(query: str, field: str | None = None) -> QueryBuilder
```

Method	Score Range	Description
`score_bm25()`	[0, +inf)	Standard BM25 relevance score
`score_bayesian_bm25()`	[0, 1]	Calibrated probability of relevance

```
results = (
    engine.query("articles")
    .term("transformer", field="title")
    .score_bm25("transformer")
    .execute()
)
for entry in results:
    print(f"doc={entry.doc_id}, score={entry.payload.score:.4f}")
```

Advanced Scoring

```
# Sparse thresholding (ReLU-as-MAP)
.sparse_threshold(threshold: float) -> QueryBuilder

# Multi-field Bayesian BM25
.score_multi_field_bayesian(query: str, fields: list[str], weights: list[float] | None = None) -> QueryBuilder

# Bayesian BM25 with external prior
.score_bayesian_with_prior(query: str, field: str | None = None, *, prior_fn: Callable) -> QueryBuilder

# Parameter learning
.learn_params(query: str, labels: list[int], *, mode: str = "balanced", field: str | None = None) -> dict[str, float]
```

Fusion

```
.fuse_log_odds(*builders: QueryBuilder, alpha: float = 0.5) -> QueryBuilder
.fuse_prob_and(*builders: QueryBuilder) -> QueryBuilder
.fuse_prob_or(*builders: QueryBuilder) -> QueryBuilder
```

Combine multiple relevance signals with calibrated probability fusion.

```
text_q = engine.query("papers").term("attention").score_bayesian_bm25("attention")
vec_q = engine.query("papers").knn(query_vec, k=20)

# Log-odds fusion
fused = engine.query("papers").fuse_log_odds(text_q, vec_q)
results = fused.execute()
```

Advanced Fusion

```python
# Attention-weighted fusion
.fuse_attention(*builders: QueryBuilder, alpha: float = 0.5) -> QueryBuilder

# Learned-weight fusion
.fuse_learned(*builders: QueryBuilder, alpha: float = 0.5) -> QueryBuilder

# Multi-stage retrieval pipeline
.multi_stage(stages: list[tuple[QueryBuilder, int | float]]) -> QueryBuilder
```

Graph Operations

```python
.traverse(start: int, label: str | None = None, max_hops: int = 1) -> QueryBuilder
.match_pattern(pattern: GraphPattern) -> QueryBuilder
.rpq(expr: str, start: int | None = None) -> QueryBuilder
.vertex_aggregate(property_name: str, agg_fn: str = "sum") -> AggregateResult
```

```python
# BFS traversal: 3-hop from vertex 1 along 'manages' edges
team = engine.query("org").traverse(1, "manages", max_hops=3).execute()

# Aggregate vertex properties
salary = engine.query("org").traverse(1, "manages", 3).vertex_aggregate("salary", "sum")
print(f"Total salary: ${salary.value:,}")

# Regular path query with Kleene star
reachable = engine.query("org").rpq("manages*", start=1).execute()
```

Temporal Graph Operations

```python
# Temporal traversal with point-in-time filter
.temporal_traverse(start: int, label: str | None = None, max_hops: int = 1, *,
                   timestamp: float | None = None, time_range: tuple[float, float] | None = None) -> QueryBuilder

# GNN message passing
.message_passing(k_layers: int = 2, aggregation: str = "mean",
                 property_name: str | None = None) -> QueryBuilder
```

Vector Exclusion

```python
.vector_exclude(negative_vector: NDArray, threshold: float) -> QueryBuilder
```

Exclude documents similar to a negative query vector.

Joins

```
.join(other: QueryBuilder, left_field: str, right_field: str) -> QueryBuilder
.vector_join(other: QueryBuilder, left_field: str, right_field: str, threshold: float) -> QueryBuilder
```

Aggregation

```
.aggregate(field: str, agg: str) -> AggregateResult
.facet(field: str) -> FacetResult
.vector_facet(field: str, query_vector: NDArray, threshold: float) -> FacetResult
```

Method	Returns	Description
`aggregate()`	`AggregateResult`	Collection-level aggregation (count, sum, avg, min, max)
`facet()`	`FacetResult`	Facet counts by field value
`vector_facet()`	`FacetResult`	Facet counts conditioned on vector similarity

```
# Per-collection aggregation
avg_price = engine.query("products").knn(q, 10).aggregate("price", "avg")
print(f"Average price: ${avg_price.value:.2f}")

# Facet counts
facets = engine.query("products").term("wireless").facet("category")
for value, count in facets.counts.items():
    print(f"  {value}: {count}")
```

Hierarchical Data

```
.path_filter(path: PathExpr, predicate: Predicate) -> QueryBuilder
.path_project(*paths: PathExpr) -> QueryBuilder
.unnest(path: PathExpr) -> QueryBuilder
.path_aggregate(path: str | PathExpr, agg: str) -> QueryBuilder
```

Operate on nested JSON-like documents.

```
from uqa.core.types import Equals, GreaterThan

# Filter by nested field
results = (
    engine.query("orders")
    .path_filter(["shipping", "city"], Equals("Seoul"))
    .execute()
)

# Aggregate nested array values
totals = (
    engine.query("orders")
    .path_aggregate("items.price", "sum")
```

```
    .execute()
)
```

Execution

```
.execute() -> PostingList                    # Execute and return PostingList
.execute_arrow() -> pa.Table                 # Execute and return PyArrow Table
.execute_parquet(path: str) -> None          # Execute and write to Parquet
.explain() -> str                            # Return optimized query plan
```

```
# Arrow export
table = engine.query("articles").term("attention").execute_arrow()
print(table.column_names)  # ['_doc_id', '_score']
print(table.num_rows)

# Parquet export
engine.query("articles").term("attention").execute_parquet("results.parquet")

# Explain plan
plan = engine.query("articles").term("attention").score_bm25("attention").explain()
print(plan)
```

---

4. Core Types

```
from uqa.core.types import (
    PostingEntry, Payload, GeneralizedPostingEntry,
    Vertex, Edge, IndexStats,
    Equals, NotEquals, GreaterThan, GreaterThanOrEqual,
    LessThan, LessThanOrEqual, Between, InSet,
    IsNull, IsNotNull, Like, NotLike, ILike, NotILike,
)
```

PostingEntry

A single entry in a posting list.

```
@dataclass(frozen=True)
class PostingEntry:
    doc_id: int
    payload: Payload
```

Payload

Carries positions, score, and field values.

```python
@dataclass(frozen=True)
class Payload:
    positions: tuple[int, ...] = ()
    score: float = 0.0
    fields: dict[str, Any] = field(default_factory=dict)
```

GeneralizedPostingEntry

Join result entry carrying a tuple of document IDs.

```python
@dataclass(frozen=True)
class GeneralizedPostingEntry:
    doc_ids: tuple[int, ...]
    payload: Payload
```

Vertex

```python
@dataclass(frozen=True)
class Vertex:
    vertex_id: int
    label: str
    properties: dict[str, Any] = field(default_factory=dict)
```

Edge

```python
@dataclass(frozen=True)
class Edge:
    edge_id: int
    source_id: int
    target_id: int
    label: str
    properties: dict[str, Any] = field(default_factory=dict)
```

IndexStats

```python
@dataclass
class IndexStats:
    total_docs: int = 0
    avg_doc_length: float = 0.0
    dimensions: int = 0
```

Predicates

All predicates implement `evaluate(value: Any) -> bool`.

Predicate	Constructor	Matches when
`Equals(target)`	`Equals(42)`	`value == target`
`NotEquals(target)`	`NotEquals(42)`	`value != target`
`GreaterThan(target)`	`GreaterThan(10)`	`value > target`
`GreaterThanOrEqual(target)`	`GreaterThanOrEqual(10)`	`value >= target`
`LessThan(target)`	`LessThan(100)`	`value < target`
`LessThanOrEqual(target)`	`LessThanOrEqual(100)`	`value <= target`
`Between(low, high)`	`Between(10, 100)`	`low <= value <= high`
`InSet(values)`	`InSet(frozenset({1,2,3}))`	`value in values`
`IsNull()`	`IsNull()`	`value is None`
`IsNotNull()`	`IsNotNull()`	`value is not None`
`Like(pattern)`	`Like("%hello%")`	SQL LIKE (case-sensitive)
`NotLike(pattern)`	`NotLike("%test%")`	SQL NOT LIKE
`ILike(pattern)`	`ILike("%hello%")`	SQL ILIKE (case-insensitive)
`NotILike(pattern)`	`NotILike("%test%")`	SQL NOT ILIKE

---

5. PostingList

The universal data structure for query results.

```
from uqa.core.posting_list import PostingList, GeneralizedPostingList
```

Boolean Algebra

```
pl1.union(pl2) -> PostingList        # A | B
pl1.intersect(pl2) -> PostingList    # A & B
pl1.difference(pl2) -> PostingList   # A - B
pl1.complement(universal) -> PostingList  # U - A
```

Operator overloads: `|` (union), `&` (intersect), `-` (difference).

Methods

Method	Description
`get_entry(doc_id)`	Retrieve entry by doc_id (binary search)
`top_k(k)`	Return top-k entries by score
`with_scores(score_fn)`	Apply scoring function to all entries
`doc_ids`	Property: set of all document IDs
`entries`	Property: list of PostingEntry

Iteration

```python
results = engine.query("articles").term("attention").execute()
for entry in results:
    print(f"doc_id={entry.doc_id}, score={entry.payload.score:.4f}")
    print(f"fields={entry.payload.fields}")
```

---

6. Text Analysis

```python
from uqa.analysis.analyzer import (
    Analyzer, DEFAULT_ANALYZER,
    standard_analyzer, whitespace_analyzer, keyword_analyzer,
    register_analyzer, get_analyzer, drop_analyzer, list_analyzers,
)
```

Analyzer

```python
Analyzer(
    tokenizer: Tokenizer | None = None,
    token_filters: list[TokenFilter] | None = None,
    char_filters: list[CharFilter] | None = None,
)
```

```python
analyzer = Analyzer(
    tokenizer=StandardTokenizer(),
    token_filters=[LowerCaseFilter(), PorterStemFilter()],
    char_filters=[HTMLStripCharFilter()],
)
tokens = analyzer.analyze("The quick brown fox")  # ['quick', 'brown', 'fox']
```

Built-in Analyzers

Four built-in analyzers are registered by default under names `"whitespace"`, `"standard"`, `"standard_cjk"`, and `"keyword"`. They cannot be overwritten or dropped.

Name	Factory	Pipeline
`"standard"`	`standard_analyzer(language="english")`	StandardTokenizer -> LowerCaseFilter -> ASCIIFoldingFilter -> StopWordFilter -> PorterStemFilter
`"whitespace"`	`whitespace_analyzer()`	WhitespaceTokenizer -> LowerCaseFilter
`"standard_cjk"`	`standard_cjk_analyzer(language="english")`	StandardTokenizer -> LowerCaseFilter -> ASCIIFoldingFilter -> StopWordFilter -> PorterStemFilter -> NGramFilter(2, 3, keep_short=True)
`"keyword"`	`keyword_analyzer()`	KeywordTokenizer (no filters)

`DEFAULT_ANALYZER` is `standard_analyzer()`. When no analyzer is assigned to a field, the default analyzer is used for both indexing and searching.

Tokenizers

```python
from uqa.analysis.tokenizer import (
    WhitespaceTokenizer, StandardTokenizer, LetterTokenizer,
    NGramTokenizer, PatternTokenizer, KeywordTokenizer,
)
```

Class	Description	Parameters
`WhitespaceTokenizer`	Split on whitespace (`str.split()`)	(none)
`StandardTokenizer`	Unicode word-boundary tokenizer (`\w+` regex)	(none)
`LetterTokenizer`	Extract runs of ASCII letters only (`[a-zA-Z]+`)	(none)
`NGramTokenizer(min_gram, max_gram)`	Split on whitespace, then generate character n-grams per word	`min_gram` (default 1), `max_gram` (default 2)
`PatternTokenizer(pattern)`	Split text using a regex delimiter	`pattern` (default `r"\W+"`)
`KeywordTokenizer`	Entire input as a single token, unmodified	(none)

Token Filters

```python
from uqa.analysis.token_filter import (
    LowerCaseFilter, StopWordFilter, PorterStemFilter,
    ASCIIFoldingFilter, SynonymFilter, NGramFilter,
    EdgeNGramFilter, LengthFilter,
)
```

Class	Description	Parameters
`LowerCaseFilter`	Convert all tokens to lowercase	(none)
`StopWordFilter(language, custom_words)`	Remove stop words from the token stream	`language` (default `"english"`), `custom_words` (optional `set[str]`)
`PorterStemFilter`	Apply the Porter stemming algorithm (suffix stripping)	(none)
`ASCIIFoldingFilter`	Fold Unicode characters to ASCII equivalents (NFKD normalization)	(none)
`SynonymFilter(synonyms, synonyms_path)`	Expand tokens with synonyms; original token kept, alternatives appended	`synonyms`: `dict[str, list[str]]` or `synonyms_path`: file path (mutually exclusive)
`NGramFilter(min_gram, max_gram, keep_short)`	Generate character n-grams from each token	`min_gram` (default 2), `max_gram` (default 3), `keep_short` (default False)
`EdgeNGramFilter(min_gram, max_gram)`	Generate prefix n-grams (autocomplete/typeahead)	`min_gram` (default 1), `max_gram` (default 20)
`LengthFilter(min_length, max_length)`	Keep tokens within a length range	`min_length` (default 0), `max_length` (default 0 = no limit)

Character Filters

```
from uqa.analysis.char_filter import (
    HTMLStripCharFilter, MappingCharFilter, PatternReplaceCharFilter,
)
```

Class	Description	Parameters
`HTMLStripCharFilter`	Strip HTML tags and convert common HTML entities	(none)
`MappingCharFilter(mapping)`	Apply string-level character replacements (longest-match-first)	`mapping`: `dict[str, str]`
`PatternReplaceCharFilter(pattern, replacement)`	Replace text matching a regular expression	`pattern`: regex string, `replacement` (default `""`)

Analyzer Registry

```
register_analyzer("my_analyzer", analyzer)    # Register globally
get_analyzer("my_analyzer")                   # Retrieve by name
drop_analyzer("my_analyzer")                  # Remove
list_analyzers()                              # List all names
```

Serialization

```
config = analyzer.to_dict()    # dict
json_str = analyzer.to_json()  # JSON string

restored = Analyzer.from_dict(config)
restored = Analyzer.from_json(json_str)
```

---

7. Storage Layer

DocumentStore

```
from uqa.storage.document_store import DocumentStore

store = DocumentStore()
store.put(1, {"name": "Alice", "age": 30})
doc = store.get(1)            # {"name": "Alice", "age": 30}
store.get_field(1, "name")    # "Alice"
store.delete(1)
len(store)                    # number of documents
store.doc_ids                 # set of all doc IDs
```

InvertedIndex

```python
from uqa.storage.inverted_index import InvertedIndex

idx = InvertedIndex(analyzer=standard_analyzer())
indexed_terms = idx.add_document(1, {"title": "hello world", "body": "..."})
pl = idx.get_posting_list("title", "hello")    # PostingList for field+term
pl = idx.get_posting_list_any_field("hello")   # PostingList across all fields
freq = idx.doc_freq("title", "hello")          # document frequency
idx.remove_document(1)
```

`InvertedIndex` supports per-field analyzer overrides:

```python
idx.set_field_analyzer("body", custom_analyzer)
analyzer = idx.get_field_analyzer("body")
```

SpatialIndex (R*Tree)

Spatial columns (`POINT`) store `[longitude, latitude]` pairs in the document store. R*Tree indexes must be created explicitly via `CREATE INDEX ... USING rtree`. Without an index, `spatial_within()` falls back to brute-force Haversine scan.

```cpp
-- Define a POINT column (storage only, no index)
CREATE TABLE places (id SERIAL PRIMARY KEY, location POINT);

-- Create an R*Tree index explicitly
CREATE INDEX idx_loc ON places USING rtree (location);

-- Insert with POINT constructor
INSERT INTO places (location) VALUES (POINT(-73.9857, 40.7484));
```

Direct Python API:

```python
from uqa.storage.spatial_index import SpatialIndex, haversine_distance

index = SpatialIndex("places", "location")
index.add(1, -73.9857, 40.7484)  # (doc_id, longitude, latitude)

# Range search: all points within 2km
results = index.search_within(-73.9973, 40.7308, 2000)  # returns PostingList

# Haversine distance
dist = haversine_distance(40.7308, -73.9973, 40.7484, -73.9857)  # meters
```

HNSWIndex (Vector Index)

Vector columns (`VECTOR(n)`) store data in the document store. HNSW indexes must be created explicitly via `CREATE INDEX ... USING hnsw`. Without an index, `knn_match()` falls back to brute-force exact cosine similarity scan.

```sql
-- Define a vector column (storage only, no index)
CREATE TABLE docs (id SERIAL PRIMARY KEY, emb VECTOR(384));

-- Create an HNSW index explicitly
CREATE INDEX idx_emb ON docs USING hnsw (emb);

-- With custom parameters
CREATE INDEX idx_emb ON docs USING hnsw (emb) WITH (ef_construction = 200, m = 16);
```

Direct Python API:

```python
from uqa.storage.vector_index import HNSWIndex
import numpy as np

index = HNSWIndex(dimensions=64, ef_construction=200, m=16)
index.add(1, np.random.randn(64).astype(np.float32))

# K-nearest neighbors
results = index.search_knn(query_vec, k=10)

# Threshold search
results = index.search_threshold(query_vec, threshold=0.8)
```

The index auto-resizes as elements are added — no capacity parameter is needed.

---

8. Graph Store

```python
from uqa.graph.store import GraphStore
from uqa.core.types import Vertex, Edge

gs = GraphStore()

# --- Named Graph Lifecycle ---
gs.create_graph("social")          # Create a named graph
gs.has_graph("social")             # True
gs.graph_names()                   # ["social"]

# Add vertices and edges (graph= keyword required)
gs.add_vertex(Vertex(1, "Person", {"name": "Alice"}), graph="social")
gs.add_vertex(Vertex(2, "Person", {"name": "Bob"}), graph="social")
gs.add_edge(Edge(1, source_id=1, target_id=2, label="KNOWS", properties={}), graph="social")

# Query (graph= keyword required)
v = gs.get_vertex(1, graph="social")
neighbors = gs.neighbors(1, label="KNOWS", direction="out", graph="social") # [2]
people = gs.vertices_by_label("Person", graph="social")  # [Vertex(1, ...), Vertex(2, ...)]

# Remove
```

```
gs.remove_vertex(1, graph="social")  # also removes incident edges
gs.remove_edge(1, graph="social")

# --- Graph Algebra ---
gs.union_graphs("combined", "graph_a", "graph_b")       # Union of two graphs
gs.intersect_graphs("shared", "graph_a", "graph_b")     # Intersection
gs.difference_graphs("only_a", "graph_a", "graph_b")    # Difference
gs.copy_graph("backup", "social")                        # Deep copy

# --- Per-Graph Statistics ---
gs.degree_distribution(graph="social")     # {vertex_id: degree}
gs.label_degree(graph="social")            # {label: avg_degree}
gs.vertex_label_counts(graph="social")     # {label: count}

# --- Drop ---
gs.drop_graph("social")
```

Graph Indexes

```
from uqa.graph.index import LabelIndex, NeighborhoodIndex, PathIndex

# Label index: fast lookup of edges by label
label_idx = LabelIndex.build(gs)
edge_ids = label_idx.edges_by_label("KNOWS")

# Neighborhood index: pre-computed k-hop neighborhoods
nbr_idx = NeighborhoodIndex.build(gs, max_hops=3)
reachable = nbr_idx.neighbors(vertex_id=1, hops=2)

# Path index: pre-computed labeled paths
path_idx = PathIndex.build(gs, label_sequences=[["KNOWS", "WORKS_AT"]])
pairs = path_idx.lookup(["KNOWS", "WORKS_AT"])  # [(source, target), ...]
```

SubgraphIndex

```
from uqa.graph.index import SubgraphIndex

# Pre-index frequent patterns
idx = SubgraphIndex.build(graph_store, [pattern1, pattern2])

# O(1) lookup
matches = idx.lookup(pattern1)  # set[frozenset[int]] or None

# Use with PatternMatchOperator (automatic cache check)
ctx = ExecutionContext(graph_store=gs, subgraph_index=idx)
result = PatternMatchOperator(pattern1).execute(ctx)  # uses cache

# Invalidation
idx.invalidate({"knows"})  # remove entries with "knows" edges
```

Incremental Pattern Matching

```python
from uqa.graph.incremental_match import GraphDelta, IncrementalPatternMatcher

matcher = IncrementalPatternMatcher(pattern, initial_matches)

# After graph mutation:
delta = GraphDelta(added_edge_ids={new_edge_id})
updated_matches = matcher.update(graph_store, delta)
```

Field	Description
`GraphDelta.added_vertex_ids`	Set of added vertex IDs
`GraphDelta.removed_vertex_ids`	Set of removed vertex IDs
`GraphDelta.added_edge_ids`	Set of added edge IDs
`GraphDelta.removed_edge_ids`	Set of removed edge IDs

Graph Patterns

```python
from uqa.graph.pattern import VertexPattern, EdgePattern, GraphPattern

pattern = GraphPattern(
    vertex_patterns=[
        VertexPattern("a", constraints=[]),
        VertexPattern("b", constraints=[]),
    ],
    edge_patterns=[
        EdgePattern("a", "b", "KNOWS", constraints=[]),
    ],
)
```

---

9. Scoring

BM25

```python
from uqa.scoring.bm25 import BM25Scorer, BM25Params

scorer = BM25Scorer(BM25Params(k1=1.5, b=0.75), index_stats)
score = scorer.score(term_freq=3, doc_length=100, doc_freq=50)
idf = scorer.idf(doc_freq=50)
combined = scorer.combine_scores([score1, score2])
```

Bayesian BM25

```python
from uqa.scoring.bayesian_bm25 import BayesianBM25Scorer, BayesianBM25Params

scorer = BayesianBM25Scorer(
    BayesianBM25Params(k1=1.5, b=0.75, alpha=0.5, beta=0.5, base_rate=0.1),
    index_stats,
)
prob = scorer.score(term_freq=3, doc_length=100, doc_freq=50)  # P(rel) in [0,1]
```

Vector Scoring

```python
from uqa.scoring.vector import VectorScorer

sim = VectorScorer.cosine_similarity(vec_a, vec_b)
prob = VectorScorer.similarity_to_probability(sim)  # map [-1,1] to [0,1]
```

Calibration Metrics

```python
from uqa.scoring.calibration import CalibrationMetrics

ece = CalibrationMetrics.ece(probabilities, labels, n_bins=10)
brier = CalibrationMetrics.brier(probabilities, labels)
report = CalibrationMetrics.report(probabilities, labels, n_bins=10)
diagram = CalibrationMetrics.reliability_diagram(probabilities, labels, n_bins=10)
```

Parameter Learner

```python
from uqa.scoring.parameter_learner import ParameterLearner

learner = ParameterLearner(alpha=1.0, beta=0.0, base_rate=0.5)
params = learner.fit(scores, labels, mode="balanced")  # batch
learner.update(score, label)  # online
current = learner.params()    # {"alpha", "beta", "base_rate"}
```

External Prior Scorer

```python
from uqa.scoring.external_prior import ExternalPriorScorer, recency_prior, authority_prior

scorer = ExternalPriorScorer(params, index_stats, prior_fn)
score = scorer.score_with_prior(term_freq, doc_length, doc_freq, doc_fields)

# Built-in prior factories
fn = recency_prior("timestamp", decay_days=30.0)
fn = authority_prior("level", levels={"high": 0.8, "medium": 0.6, "low": 0.4})
```

Multi-Field Scorer

```python
from uqa.scoring.multi_field import MultiFieldBayesianScorer

scorer = MultiFieldBayesianScorer(
    [("title", BayesianBM25Params(), 2.0), ("body", BayesianBM25Params(), 1.0)],
    index_stats,
)
score = scorer.score_document(doc_id, tf_per_field, dl_per_field, df_per_field)
```

---

10. Query Planning

Optimizer

```python
from uqa.planner.optimizer import QueryOptimizer

optimizer = QueryOptimizer(index_stats)
optimized_op = optimizer.optimize(operator_tree)
```

Optimization passes:

- `_simplify_algebra()` -- idempotent intersection/union, absorption law, empty elimination
- Filter pushdown (into scans, intersect children, graph patterns)
- Vector threshold merge
- Intersect reordering by cardinality

Cost Model

```python
from uqa.planner.cost_model import CostModel

model = CostModel()
cost = model.estimate(operator, index_stats)
```

Cardinality Estimator

```python
from uqa.planner.cardinality import CardinalityEstimator

estimator = CardinalityEstimator()
card = estimator.estimate(operator, index_stats)
```

Join Order Optimizer (DPccp)

```python
from uqa.planner.join_order import JoinOrderOptimizer

optimizer = JoinOrderOptimizer()
join_op, table = optimizer.optimize(relations, predicates)
```

The DPccp algorithm enumerates connected subgraph complement pairs to find the optimal join order. For small cardinalities (<= 100), `IndexJoinOperator` (binary search) is selected; otherwise `InnerJoinOperator` (hash join with build-on-smaller-side) is used.

Plan Executor

```
from uqa.planner.executor import PlanExecutor

executor = PlanExecutor(execution_context)
result = executor.execute(optimized_op)
plan_str = executor.explain(optimized_op)
```

---

11. Operators

All operators implement `execute(context: ExecutionContext) -> PostingList`.

Primitive Operators

```
from uqa.operators.primitive import (
    TermOperator, VectorSimilarityOperator, KNNOperator,
    SpatialWithinOperator, FilterOperator, FacetOperator, ScoreOperator,
)
```

Operator	Description
`TermOperator(term, field)`	Retrieve posting list for a term
`VectorSimilarityOperator(query, threshold, field)`	Vector similarity >= threshold
`KNNOperator(query, k, field)`	Top-k nearest neighbors
`SpatialWithinOperator(field, cx, cy, dist)`	Spatial range query (R*Tree + Haversine)
`FilterOperator(field, predicate, source)`	Filter entries by predicate
`FacetOperator(field, source)`	Compute facet counts
`ScoreOperator(scorer, source, terms)`	Apply scoring function

Boolean Operators

```
from uqa.operators.boolean import (
    UnionOperator, IntersectOperator, ComplementOperator,
)
```

Operator	Description
`UnionOperator(sources)`	Union of posting lists
`IntersectOperator(sources)`	Intersection of posting lists
`ComplementOperator(source)`	Complement of posting list

Graph Operators

```
from uqa.graph.operators import (
    TraverseOperator, PatternMatchOperator, RegularPathQueryOperator,
    VertexAggregationOperator,
)
```

Operator	Description
`TraverseOperator(start, label, max_hops, score=DEFAULT_GRAPH_SCORE)`	BFS graph traversal
`PatternMatchOperator(pattern, score=DEFAULT_GRAPH_SCORE)`	Subgraph isomorphism
`RegularPathQueryOperator(path_expr, start, score=DEFAULT_GRAPH_SCORE)`	NFA-based path query
`VertexAggregationOperator(source, property, agg_fn)`	Aggregate vertex properties

`DEFAULT_GRAPH_SCORE` is 0.9 (defined in `uqa.graph.operators`). Reachable vertices receive this score in their payload, which integrates with log-odds fusion.

Graph Centrality Operators

```
from uqa.graph.centrality import (
    PageRankOperator,
    HITSOperator,
    BetweennessCentralityOperator,
)

# PageRank: power iteration centrality
pr = PageRankOperator(damping=0.85, max_iterations=100, tolerance=1e-6)
results = pr.execute(ctx)  # GraphPostingList, score = normalized PageRank

# HITS: hub/authority scoring
hits = HITSOperator(max_iterations=100, tolerance=1e-6)
results = hits.execute(ctx)
# entry.payload.fields["hub_score"], entry.payload.fields["authority_score"]
# entry.payload.score = authority_score

# Betweenness centrality: Brandes algorithm
bc = BetweennessCentralityOperator()
```

```
results = bc.execute(ctx)  # score = normalized betweenness in [0, 1]
```

Parameter	`PageRankOperator`	`HITSOperator`	`BetweennessCentralityOperator`
`damping`	0.85	N/A	N/A
`max_iterations`	100	100	N/A
`tolerance`	1e-6	1e-6	N/A

Bounded RPQ

```
from uqa.graph.pattern import parse_rpq, BoundedLabel

# Parse bounded repetition: e{min,max}
expr = parse_rpq("knows{2,4}")  # 2 to 4 hops
assert isinstance(expr, BoundedLabel)
assert expr.min_hops == 2
assert expr.max_hops == 4
```

Weighted Path Query

```
from uqa.graph.operators import WeightedPathQueryOperator

op = WeightedPathQueryOperator(
    path_expr=parse_rpq("knows/knows"),
    weight_property="weight",    # edge property to accumulate
    aggregate_fn="sum",          # "sum", "max", or "min"
    predicate=lambda w: w > 5.0, # filter at accepting states
    start_vertex=1,
)
results = op.execute(ctx)
# entry.payload.fields["path_weight"] = cumulative weight
```

Advanced Scoring Operators

Operator	Constructor	Description
`SparseThresholdOperator`	`(source, threshold)`	ReLU thresholding: max(0, score - threshold)
`MultiFieldSearchOperator`	`(fields, query, weights)`	Multi-field Bayesian BM25 with log-odds fusion

Advanced Fusion Operators

Operator	Constructor	Description
`AttentionFusionOperator`	`(signals, attention, query_features)`	Attention-weighted log-odds conjunction
`LearnedFusionOperator`	`(signals, learned)`	Learned-weight log-odds conjunction
`MultiStageOperator`	`(stages)`	Cascading (operator, cutoff) retrieval pipeline

ProgressiveFusionOperator

```python
from uqa.operators.progressive_fusion import ProgressiveFusionOperator

op = ProgressiveFusionOperator(
    stages=[
        ([signal1, signal2], 50),  # Stage 1: fuse sig1+sig2, keep top-50
        ([signal3], 10),           # Stage 2: add sig3, keep top-10
    ],
    alpha=0.5,
    gating="relu",  # optional
)
result = op.execute(ctx)
```

Each stage accumulates signals and narrows candidates via `FusionWANDScorer` top-k pruning.

Temporal Graph Operators

Operator	Constructor	Description
`TemporalTraverseOperator`	`(start, label, max_hops, temporal_filter)`	BFS with temporal edge filtering
`TemporalPatternMatchOperator`	`(pattern, temporal_filter)`	Pattern match with temporal edge constraints

GNN Operators

Operator	Constructor	Description
`MessagePassingOperator`	`(k_layers, aggregation, property_name)`	K-layer neighbor aggregation + sigmoid
`GraphEmbeddingOperator`	`(dimensions, k_layers)`	Structural vertex embeddings

Graph Maintenance

Class	Description
`GraphDelta`	Records mutation operations; `add_vertex()`, `remove_vertex()`, `add_edge()`, `remove_edge()`, `affected_vertex_ids()`, `affected_edge_labels()`
`VersionedGraphStore`	`__init__(base)`, `apply(delta)`, `rollback(to_version)`, `version`, `on_invalidate(callback)`
`TemporalFilter`	`__init__(timestamp=None, time_range=None)`, `is_valid(properties)`

Fusion Classes

Class	Description
`AttentionFusion`	`__init__(n_signals, n_query_features, alpha)`, `fuse(probs, query_features)`, `fit()`, `update()`, `state_dict()`, `load_state_dict()`
`LearnedFusion`	`__init__(n_signals, alpha)`, `fuse(probs)`, `fit()`, `update()`, `state_dict()`, `load_state_dict()`
`QueryFeatureExtractor`	`__init__(inverted_index)`, `extract(query_terms, field)`, `n_features` (= 6)
`FusionWANDScorer`	`__init__(signal_posting_lists, signal_upper_bounds, alpha, k)`, `score_top_k()`

Fusion Gating

```
from uqa.fusion.log_odds import LogOddsFusion

# ReLU gating
f = LogOddsFusion(confidence_alpha=0.5, gating="relu")
result = f.fuse([0.8, 0.6, 0.7])

# Swish gating
f = LogOddsFusion(confidence_alpha=0.5, gating="swish")
```

The `gating` parameter is also available on `LogOddsFusionOperator` and `FusionWANDScorer`.

`fuse_mean(probabilities: list[float]) -> float`

Log-odds mean aggregation (Definition 4.1.1, Paper 4). Computes the arithmetic mean in log-odds space and maps back to probability via sigmoid. Unlike `fuse()`, no confidence scaling (`n^alpha`) is applied. Scale-neutral: if all signals report the same probability `p`, the output is exactly `p` regardless of `n`. This is the normalized Logarithmic Opinion Pool (Theorem 4.1.2a).

```
f = LogOddsFusion(confidence_alpha=0.5)
p = f.fuse_mean([0.8, 0.6, 0.7])  # mean in log-odds space
```

Hierarchical Operators

```
from uqa.operators.hierarchical import (
    PathFilterOperator, PathProjectOperator,
    PathUnnestOperator, PathAggregateOperator,
)
```

All hierarchical operators expose a cost estimation method:

Method	Returns	Description
`cost_estimate(stats: IndexStats)`	`float`	Estimated execution cost based on index statistics

Aggregation Monoids

```
from uqa.operators.aggregation import (
    CountMonoid, SumMonoid, AvgMonoid,
    MinMonoid, MaxMonoid, QuantileMonoid,
)
```

Monoid	`identity()`	`finalize()`	Description
`CountMonoid`	`0`	`int`	Count of accumulated values
`SumMonoid`	`0.0`	`float`	Sum of values
`AvgMonoid`	`(0.0, 0)`	`float`	Arithmetic mean (sum, count)
`MinMonoid`	`float('inf')`	`float`	Minimum value
`MaxMonoid`	`float('-inf')`	`float`	Maximum value
`QuantileMonoid(quantile=0.5)`	`list[float]`	`float`	Quantile via linear interpolation

All monoids implement `accumulate(state, value)` and `combine(state_a, state_b)` for parallel-safe aggregation.

`QuantileMonoid(quantile: float = 0.5)`

Collects values and computes the requested quantile at finalize using linear interpolation between adjacent values. `quantile=0.5` gives the median, `quantile=0.95` gives P95.

```
median = QuantileMonoid(quantile=0.5)
p95 = QuantileMonoid(quantile=0.95)
```

Method	Signature	Description
`identity()`	`-> list[float]`	Returns empty list
`accumulate()`	`(state: list[float], value: float) -> list[float]`	Appends value to state
`combine()`	`(state_a: list[float], state_b: list[float]) -> list[float]`	Concatenates two states
`finalize()`	`(state: list[float]) -> float`	Sorts and interpolates at the requested quantile

Join Operators

```python
from uqa.joins.inner import InnerJoinOperator
from uqa.joins.index import IndexJoinOperator
from uqa.joins.base import JoinCondition

condition = JoinCondition(left_field="id", right_field="fk")
join = InnerJoinOperator(left_source, right_source, condition)
result = join.execute(context)
```

Operator	Complexity	Description
`InnerJoinOperator`	O(N + M)	Hash join, builds on smaller side
`IndexJoinOperator`	O(N log M)	Binary search join

Execution Context

```python
from uqa.operators.base import ExecutionContext

context = ExecutionContext(
    document_store=doc_store,
    inverted_index=inv_idx,
    vector_indexes={"embedding": hnsw_index},
    spatial_indexes={"location": spatial_index},
    graph_store=graph_store,
)
```

Field	Type	Description
`document_store`	`DocumentStore`	Document storage
`inverted_index`	`InvertedIndex`	Inverted index for term lookups
`vector_indexes`	`dict[str, HNSWIndex]`	Named vector indexes
`spatial_indexes`	`dict[str, SpatialIndex]`	Named spatial indexes
`graph_store`	`GraphStore | None`	Graph store for traversal operators
`path_index`	`PathIndex | None`	Pre-computed path index for RPQ acceleration
`subgraph_index`	`SubgraphIndex | None`	Pre-indexed subgraph patterns for PatternMatchOperator cache

---

12. Transaction

```python
from uqa.storage.transaction import Transaction
```

Methods

Method	Description
`commit()`	Commit the transaction
`rollback()`	Rollback the transaction
`savepoint(name)`	Create a savepoint
`release_savepoint(name)`	Release a savepoint
`rollback_to(name)`	Rollback to a savepoint

Properties

Property	Type	Description
`active`	`bool`	Whether transaction is active

Usage

```python
with engine.begin() as tx:
    engine.sql("INSERT INTO accounts (id, balance) VALUES (1, 1000)")
    tx.savepoint("sp1")
    engine.sql("UPDATE accounts SET balance = 500 WHERE id = 1")
    tx.rollback_to("sp1")  # undo the update
    # commits on exit
```

---

13. Table Schema

```python
from uqa.sql.table import Table, ColumnDef, ColumnStats, ForeignKeyDef
```

ColumnDef

```python
@dataclass
class ColumnDef:
    name: str
    type_name: str
    python_type: type
    primary_key: bool = False
    not_null: bool = False
    auto_increment: bool = False
    default: Any = None
    vector_dimensions: int | None = None
    unique: bool = False
    numeric_precision: int | None = None
    numeric_scale: int | None = None
```

ColumnStats

Column-level statistics collected by `ANALYZE`:

```python
@dataclass
class ColumnStats:
    distinct_count: int = 0
    null_count: int = 0
    min_value: Any = None
    max_value: Any = None
    row_count: int = 0
    histogram: list[Any] = []
    mcv_values: list[Any] = []
    mcv_frequencies: list[float] = []
```

ForeignKeyDef

```python
@dataclass
class ForeignKeyDef:
    column: str
    ref_table: str
    ref_column: str
```

14. Foreign Data Wrappers

Foreign Data Wrappers (FDW) let UQA query external data sources -- Parquet files, CSV files, S3 objects, DuckDB databases, and Arrow Flight SQL services -- as if they were ordinary SQL tables.

```python
from uqa.fdw.foreign_table import ForeignServer, ForeignTable, FDWPredicate
from uqa.fdw.handler import FDWHandler
```

ForeignServer

```python
@dataclass(frozen=True, slots=True)
class ForeignServer:
    name: str
    fdw_type: str              # "duckdb_fdw" or "arrow_fdw"
    options: dict[str, str]    # Connection parameters
```

Create a server with SQL:

```sql
-- DuckDB in-process (Parquet, CSV, S3, attached databases)
CREATE SERVER warehouse FOREIGN DATA WRAPPER duckdb_fdw
    OPTIONS (database ':memory:');

-- With S3 credentials
CREATE SERVER s3_lake FOREIGN DATA WRAPPER duckdb_fdw
    OPTIONS (
        database ':memory:',
        s3_region 'us-east-1',
        s3_access_key_id 'AKIA...',
        s3_secret_access_key '...'
    );

-- Arrow Flight SQL (Dremio, DataFusion, etc.)
CREATE SERVER analytics FOREIGN DATA WRAPPER arrow_fdw
    OPTIONS (host 'dremio.example.com', port '8815', tls 'true',
             username 'user', password 'secret');
```

Server Option	FDW Type	Description
`database`	`duckdb_fdw`	Path to DuckDB file. `":memory:"` for in-process.
`extensions`	`duckdb_fdw`	Comma-separated DuckDB extensions to load.
`s3_region`	`duckdb_fdw`	AWS S3 region.
`s3_access_key_id`	`duckdb_fdw`	AWS access key.
`s3_secret_access_key`	`duckdb_fdw`	AWS secret key.
`host`	`arrow_fdw`	Flight SQL server hostname.
`port`	`arrow_fdw`	Flight SQL server port (default `8815`).
`tls`	`arrow_fdw`	`"true"` to enable TLS.
`username`	`arrow_fdw`	Authentication username.
`password`	`arrow_fdw`	Authentication password.

ForeignTable

```python
@dataclass(slots=True)
class ForeignTable:
    name: str
    server_name: str
    columns: OrderedDict[str, ColumnDef]
    options: dict[str, str]
```

Create a foreign table with SQL:

```sql
-- Parquet file
CREATE FOREIGN TABLE events (
    event_id INTEGER,
    user_id  INTEGER,
    ts       TIMESTAMP,
    payload  TEXT
) SERVER warehouse OPTIONS (source '/data/events.parquet');

-- CSV file
CREATE FOREIGN TABLE logs (
    ts      TEXT,
    level   TEXT,
    message TEXT
) SERVER warehouse OPTIONS (source '/data/logs.csv');

-- S3 path
CREATE FOREIGN TABLE clicks (
    click_id INTEGER,
    url      TEXT,
    ts       TIMESTAMP
```

```sql
) SERVER s3_lake OPTIONS (source 's3://bucket/clicks/*.parquet');

-- Hive-partitioned directory
CREATE FOREIGN TABLE partitioned_events (
    event_id INTEGER,
    user_id  INTEGER,
    ts       TIMESTAMP,
    year     INTEGER,
    month    INTEGER
) SERVER warehouse
  OPTIONS (source '/data/events/', hive_partitioning 'true');

-- Arrow Flight SQL remote table
CREATE FOREIGN TABLE remote_sales (
    sale_id INTEGER,
    amount  DOUBLE PRECISION
) SERVER analytics OPTIONS (source 'public.sales');

-- Arrow Flight SQL with custom query
CREATE FOREIGN TABLE recent_sales (
    sale_id INTEGER,
    amount  DOUBLE PRECISION
) SERVER analytics OPTIONS (query 'SELECT * FROM sales WHERE year = 2026');
```

Table Option	Description
`source`	DuckDB expression (`read_parquet(...)`, file path, attached table) or remote table name for Flight SQL. Bare file paths are auto-wrapped in the appropriate `read_*()` function.
`query`	(Arrow Flight SQL only) Full SQL query sent to the remote server. Takes precedence over `source`.
`hive_partitioning`	`"true"` to enable Hive-style `key=value` partition discovery.

FDWPredicate

```python
@dataclass(frozen=True, slots=True)
class FDWPredicate:
    column: str
    operator: str    # =, !=, <>, <, <=, >, >=, IN, LIKE, NOT LIKE, ILIKE, NOT ILIKE
    value: Any       # Scalar or tuple (for IN)
```

FDW predicates are extracted automatically from WHERE clauses and pushed down to handlers for server-side filtering (e.g. Hive partition pruning).

FDWHandler

```python
class FDWHandler(ABC):
    @abstractmethod
    def scan(
        self,
        foreign_table: ForeignTable,
        columns: list[str] | None = None,
        predicates: list[FDWPredicate] | None = None,
        limit: int | None = None,
    ) -> pa.Table: ...

    @abstractmethod
    def close(self) -> None: ...
```

Built-in handlers:

Handler	FDW Type	Description
`DuckDBFDWHandler`	`duckdb_fdw`	In-process DuckDB. Parquet, CSV, JSON, S3, attached DBs.
`ArrowFlightSQLFDWHandler`	`arrow_fdw`	Remote Arrow Flight SQL client (Dremio, DataFusion, etc.).

Full Query Pushdown

UQA automatically delegates entire SQL queries to the foreign data source when possible, rather than scanning raw rows into the UQA pipeline. This eliminates Python-side materialization and lets the external engine handle joins, aggregation, window functions, subqueries, and sorting natively.

How it works. Before building a UQA operator tree, the compiler inspects every table referenced in the SELECT statement (FROM, JOINs, subqueries, CTEs). If all tables resolve to foreign tables on a single server, the original SQL is deparsed from the AST and sent directly to that server. The result comes back as a PyArrow table and is converted to an `SQLResult` -- the UQA operator pipeline is never constructed.

```
SELECT with only foreign tables
        |
        v
  _try_foreign_full_pushdown()
        |
        +-- all tables on same duckdb_fdw server? --> execute on DuckDB
        |
        +-- all tables on same arrow_fdw server?   --> execute via Flight SQL
        |
        +-- otherwise                              --> fall through to UQA pipeline
```

Pure foreign queries -- when every table in the query belongs to the same foreign server -- are pushed down with no restrictions:

```
engine = Engine()

# Set up DuckDB FDW with two Parquet files
engine.sql("CREATE SERVER wh FOREIGN DATA WRAPPER duckdb_fdw")
engine.sql("""
    CREATE FOREIGN TABLE orders (
        order_id INTEGER, product_id INTEGER,
        customer TEXT, quantity INTEGER
    ) SERVER wh OPTIONS (source '/data/orders.parquet')
""")
engine.sql("""
    CREATE FOREIGN TABLE products (
        id INTEGER, name TEXT, price DOUBLE PRECISION
    ) SERVER wh OPTIONS (source '/data/products.parquet')
""")

# This entire query is pushed to DuckDB -- including the JOIN,
# GROUP BY, HAVING, and ORDER BY:
result = engine.sql("""
    SELECT p.name, SUM(o.quantity) AS total_qty
    FROM orders o
    INNER JOIN products p ON o.product_id = p.id
    GROUP BY p.name
    HAVING SUM(o.quantity) > 10
    ORDER BY total_qty DESC
""")
```

Window functions, CTEs, scalar subqueries, DISTINCT, LIMIT/OFFSET, and all other standard SQL clauses are also pushed down:

```
result = engine.sql("""
    WITH ranked AS (
        SELECT name, quantity,
               ROW_NUMBER() OVER (ORDER BY quantity DESC) AS rn
        FROM orders o JOIN products p ON o.product_id = p.id
    )
    SELECT name, quantity FROM ranked WHERE rn <= 5
""")
```

Mixed Foreign-Local Queries

When a query joins foreign tables with local UQA tables (e.g. a small dimension table), the local tables are shipped to DuckDB as temporary tables, and the entire query is still executed by DuckDB:

```
# Local dimension table
engine.sql("CREATE TABLE regions (region_id INTEGER PRIMARY KEY, name TEXT)")
engine.sql("""
    INSERT INTO regions (region_id, name)
    VALUES (1, 'North America'), (2, 'Europe'), (3, 'Asia')
""")

```

```python
# Foreign fact table on Parquet
engine.sql("""
    CREATE FOREIGN TABLE sales (
        sale_id INTEGER, region_id INTEGER,
        amount DOUBLE PRECISION
    ) SERVER wh OPTIONS (source '/data/sales.parquet')
""")

# The local 'regions' table is materialized into DuckDB as a temp table,
# and the full join is executed by DuckDB:
result = engine.sql("""
    SELECT r.name AS region, SUM(s.amount) AS revenue
    FROM sales s
    INNER JOIN regions r ON s.region_id = r.region_id
    GROUP BY r.name
    ORDER BY revenue DESC
""")
```

Constraints for mixed queries:

- All foreign tables must belong to the same `duckdb_fdw` server. Mixed-server queries are not pushed down.
- Arrow Flight SQL (`arrow_fdw`) does not support mixed queries -- all tables must be foreign.
- Local tables are shipped only when their row count does not exceed 100,000 rows. Larger local tables cause the query to fall through to the UQA pipeline.
- POINT columns in local tables are expanded into two DOUBLE columns (`col_lon`, `col_lat`) when shipped to DuckDB.

Automatic Fallback

If full query pushdown cannot be applied, or if DuckDB raises an error during execution (e.g. the query uses a UQA-specific function that DuckDB does not recognize), the compiler silently falls back to the standard UQA operator pipeline. Individual foreign table scans still benefit from predicate pushdown at the scan level.

The fallback triggers in any of these situations:

Condition	Behavior
Tables span multiple foreign servers	UQA pipeline with per-table FDW scans
Query references CTEs or views that do not resolve to known tables	UQA pipeline
Local table exceeds 100,000 rows (mixed query)	UQA pipeline
Arrow Flight SQL with local tables	UQA pipeline
DuckDB execution error (e.g. `spatial_within()`, `text_match()`)	UQA pipeline
EXPLAIN mode	UQA pipeline (plan tree required)

```
# This query uses a UQA-specific function. Full pushdown is attempted
# but DuckDB does not know spatial_within(), so the query automatically
# falls back to the UQA pipeline:
result = engine.sql("""
    SELECT s.sale_id, s.amount
    FROM sales s
    JOIN regions r ON s.region_id = r.region_id
    WHERE spatial_within(s.location, 'POLYGON(...)')
""")
```

Even when full pushdown is not possible, the `_ForeignTableScanOperator` still pushes simple `column op constant` predicates from the WHERE clause to the FDW handler, enabling server-side filtering (e.g. Hive partition pruning):

```
# Full pushdown fails because of text_match(), but the year >= 2025
# predicate is still pushed to DuckDB at the scan level:
result = engine.sql("""
    SELECT * FROM partitioned_events
    WHERE year >= 2025 AND text_match(payload, 'error')
""")
```

Drop Statements

```
DROP FOREIGN TABLE events;
DROP FOREIGN TABLE IF EXISTS events;

DROP SERVER warehouse;
DROP SERVER IF EXISTS warehouse;
```

Dropping a server that still has dependent foreign tables raises an error. Drop the foreign tables first.

15. Complete Example

```python
from uqa.engine import Engine
from uqa.core.types import Vertex, Edge, Equals, GreaterThan
import numpy as np

# Create engine with persistence
engine = Engine(db_path="research.db")

# --- SQL: Create tables and insert data ---
engine.sql("""
    CREATE TABLE papers (
        id SERIAL PRIMARY KEY,
        title TEXT,
        abstract TEXT,
        year INTEGER,
        embedding VECTOR(64)
    )
""")

engine.sql("""
    INSERT INTO papers (title, abstract, year)
    VALUES ('Attention Is All You Need', 'The transformer architecture...', 2017)
""")

# --- SQL: Full-text search with BM25 ---
result = engine.sql("""
    SELECT title, _score FROM papers
    WHERE text_match(title, 'transformer attention')
    ORDER BY _score DESC
    LIMIT 10
""")

# --- SQL: Hybrid search with fusion ---
result = engine.sql("""
    SELECT title, _score FROM papers
    WHERE fuse_log_odds(
        text_match(title, 'attention'),
        knn_match(embedding, $1, 5)
    )
    ORDER BY _score DESC
""", [np.random.randn(64).tolist()])

# --- Fluent API: Term search + BM25 scoring ---
results = (
    engine.query("papers")
    .term("attention", field="abstract")
    .score_bm25("attention")
    .execute()
)
for entry in results:
    print(f"  [{entry.doc_id}] score={entry.payload.score:.4f}")

```

```python
# --- Fluent API: KNN + filter ---
results = (
    engine.query("papers")
    .knn(np.random.randn(64).astype(np.float32), k=5)
    .filter("year", GreaterThan(2020))
    .execute()
)

# --- Fluent API: Graph traversal ---
engine.add_graph_vertex(Vertex(1, "Paper", {"title": "Attention"}), "papers")
engine.add_graph_vertex(Vertex(2, "Paper", {"title": "BERT"}), "papers")
engine.add_graph_edge(Edge(1, 1, 2, "cites", {}), "papers")

cited = engine.query("papers").traverse(1, "cites", max_hops=2).execute()

# --- Cypher integration ---
result = engine.sql("""
    SELECT * FROM cypher('
        MATCH (p:Paper)-[:cites]->(cited:Paper)
        RETURN cited.title AS title
    ')
""")

# --- Export to Arrow/Parquet ---
arrow_table = result.to_arrow()
result.to_parquet("citations.parquet")

engine.close()
```